
**UNIVERSITATEA SAPIENTIA DIN CLUJ-NAPOCA
FACULTATEA DE ȘTIINȚE TEHNICE ȘI UMANISTE,
TÎRGU-MUREȘ
PROGRAMUL DE STUDII ...**

**TITLUL PROIECTULUI DE
DIPLOMĂ**

PROIECT DE DIPLOMĂ

Coordonator științific:

Absolvent:

2020

UNIVERSITATEA “SAPIENTIA” din CLUJ-NAPOCA Facultatea de Științe Tehnice și Umaniste din Târgu Mureș Specializarea: ...		Viza facultății:
LUCRARE DE DIPLOMĂ		
Coordonator științific:	Candidat: Anul absolvirii:	
a) Tema lucrării de licență:		
b) Problemele principale tratate:		
c) Desene obligatorii:		
d) Softuri obligatorii:		
e) Bibliografia recomandată:		
f) Termene obligatorii de consultații: săptămânal		
g) Locul și durata practicii: Universitatea Sapientia, Facultatea de Științe Tehnice și Umaniste din Târgu Mureș Primit tema la data de: Termen de predare:		
Semnătura Director Departament	Semnătura coordonatorului	
Semnătura responsabilului programului de studiu	Semnătura candidatului	

Declarație

Subsemnatul/a ... , absolvent al specializării ..., promoția ... cunoscând prevederile Legii Educației Naționale 1/2011 și a Codului de etică și deontologie profesională a Universității Sapienția cu privire la furt intelectual declar pe propria răspundere că prezenta lucrare de licență/proiect de diplomă/disertație se bazează pe activitatea personală, cercetarea/proiectarea este efectuată de mine, informațiile și datele preluate din literatura de specialitate sunt citate în mod corespunzător.

Târgu Mureș,

Data:

Extras

Extract

Cuvinte cheie:

**SAPIENTIA ERDÉLYI MAGYAR
TUDOMÁNYEGYETEM
MAROSVÁSÁRHELYI KAR
SZÁMÍTÁSTECHNIKA SZAK**

DOLGOZAT CÍME

DIPLOMADOLGOZAT

Témavezető:

Végzős hallgató:

2020

Kivonat

Napjainkban rendkívül elterjedté váltak a webes alkalmazások, amelyeket nap mint nap használunk: akár a munkánk során, akár kapcsolattartásra, akár szórakozásra. Ez a hatalmas növekedés a webes alkalmazások használatában annak köszönhető, hogy egy böngészőből használható alkalmazás már-már az asztali programok színvonalát és funkcionalitását is meg tudja közelíteni. A modern webes alkalmazások mögött rengeteg tervezés, döntés, fejlesztés, valamint technikai vagy akár pénzügyi akadály áll, amelyek megoldásáért és kivitelezéséért egy csapat felelős. Mindenkinek megvan a szerepe az ötlet kigondolásától a UI (felhasználói felület) tervezésén át egészen a fejlesztésig és karbantartásig. A dolgozat egy ilyen webes alkalmazás tervezési folyamatait kívánja bemutatni: hogy miként jutunk el magától az ötlettől egy kész termékig, és milyen folyamatok mennek végbe még azelőtt, hogy a fejlesztők leírnának egy sor kódot, vagy a UX-designerek akár egy vonalat is húznának. Ennek demonstrálásához bemutatok egy „fiktív” fejlesztői csapatot, akik egy filmnaplózó weboldalt terveznek és fejlesztenek.

Kulcsszavak: webes alkalmazások, tervezés, fejlesztés, UX-design, filmnaplózó

Abstract

Abstract

Keywords:

Tartalomjegyzék

1. Bevezető	1
1.1. Web alkalmazások kezdetektől napjainkig	1
1.2. Korai világháló	2
1.3. Dinamikus világháló	2
1.4. Fejlett kliensoldali web (AJAX, SPA-k, keretrendszerek)	3
1.5. Mobil és web standardok (reszponzív dizájn)	3
1.6. Jelenlegi állapot és trendek	4
1.7. A web fejlődésének rövid története - idővonal	4
2. Szakirodalom áttekintése	7
2.1. Cím 1	7
2.1.1. Alcím 1	7
3. Elméleti áttekintés	9
3.1. Elméleti áttekintés	9
4. Rendszer specifikációi	11
4.1. Cím 1	11
5. Gyakorlati megvalósítás	12
5.1. Ágensek vezérlése	12
5.1.1. Potenciálmező navigáció	12

6. Eredmények	14
6.1. Cím 1	14
7. Összefoglalás	15
7.1. Összefoglalás	15
Irodalomjegyzék	15
A. Függelék	21
A.1. Alfejezet	21
A.1.1. Cím	21

Ábrák jegyzéke

2.1. V-REP, ARGoS, Gazebo összehasonlítása	7
6.1. 30 követő ágens, egy vezér	14

1. fejezet

Bevezető

1.1. Web alkalmazások kezdetektől napjainkig

A világháló (WWW) használatának kezdetét az 1989–1991 közötti periódusra datálják. Kezdeti úttörője Tim Berners-Lee volt, a CERN egyik munkatársa, aki a web alapvető komponensei közé tartozó HTTP-t (HyperText Transfer Protocol), HTML-t (HyperText Markup Language), URL-t, valamint az első webböngészőt és webszervert is megvalósította [1]. Az ekkor készült weboldalak főként statikusak voltak, majd a szerveroldali programozás és az adatbázisok használata lehetővé tette a dinamikus tartalom előállítását [2]. A 2000-es évektől a kliensoldali interaktivitás az AJAX és a böngészőoldali JavaScript használatával vált egyre hangsúlyosabbá [3]. A mobileszközök elterjedése a responzív dizájnt tette alapvető megközelítéssé [4], a skálázhatóságot pedig a felhőalapú számítástechnika segítette [5].

A modern fejlesztési folyamatok manapság nagy hangsúlyt fektetnek a DevOps szemléletre, a folyamatos integrációra, szállításra és telepítésre (CI/CD), az automatizált tesztelésre, valamint a konténerizációra [6]. Architektúráis oldalon a felhőalapú és mikroszolgáltatás-alapú megoldások terjedése vált meghatározóvá [5, 7].

1.2. Korai világháló

A világháló indulása a CERN-hez kötődik: Tim Berners-Lee 1989 márciusában írta meg az első webes javaslatot, majd 1990 végére megvalósította a HTML, a HTTP és az URL alapjait, valamint az első webszervert és böngésző/szerkesztő programot [1, 8]. Az első weboldal és információs szerver az info.cern.ch címen futott, egy CERN-ben használt NeXT számítógépen [8]. Ezek a korai oldalak főként statikus dokumentumok voltak: a webszerver a tárolt HTML-fájlokat küldte vissza a böngészőnek, dinamikus tartalomgenerálás nélkül [9]. A grafikus böngészők közül az NCSA Mosaic 1993-as megjelenése, majd a Mosaic Communications Corp. (később Netscape) 1994-es megalakulása jelentős szerepet játszott a web népszerűsödésében [8, 10]. Ezzel párhuzamosan a HTML is folyamatosan fejlődött: a HTML 4.0 1997-ben, a HTML5 pedig 2014-ben lett W3C Recommendation, miközben a HTML alapvető szerepe továbbra is a webes dokumentumok szerkezetének leírása maradt [11, 12].

1.3. Dinamikus világháló

1993-ban az NCSA (National Center for Supercomputing Applications) kifejlesztette a CGI-t (Common Gateway Interface), ezzel lehetővé téve, hogy a webszerverek külső programokat futtaszanak (például Perl-szkripteket), és menet közben oldalakat generáljanak [13]. Ez volt az első nagy választóvonal a statikus és a dinamikus weboldalak között. A szerveroldali nyelvek (PHP, Python, ASP, Java) adatbázissal párosítva működtették a korai dinamikus oldalakat (fórumokat, online üzleteket) [2, 14]. Ezeket a szerver által futtatott oldalakat minden egyes kérésnél újra kellett tölteni, ám a 2000-es évek végén az AJAX széles körű ismeretének és használatának köszönhetően az oldalak frissülni tudtak anélkül, hogy teljesen újra kellett volna őket tölteni, így gördülékenyebb felhasználói felületet biztosítottak [3, 15]. Ugyanakkor ezeknek az oldalaknak az alapja továbbra is a kliens és a szerver közötti kérés–válasz modellt követte [16].

1.4. Fejlett kliensoldali web (AJAX, SPA-k, keretrendszerek)

A 2000-es évek közepétől a webalkalmazások egyre inkább „alkalmazásszerűvé” váltak [17]. Az AJAX (a kifejezést 2005-ben vezették be) lehetővé tette, hogy a böngészőben futó JavaScript aszinkron módon kérjen le adatokat [3, 15]. Az olyan keretrendszerek, mint a jQuery (2006), Angular (2010), React (2013), Vue stb., lehetővé tették az egyoldalas alkalmazások (SPA-k) létrehozását, ahol a tartalom dinamikusan töltődik be kliensoldalon [18, 17]. Ezek az alkalmazások gyakran REST vagy GraphQL API-kkal kommunikálnak [19, 20].

A HTML5 (W3C ajánlás, 2014 október) szabványosított új API-kat (Canvas, WebSockets, Web Storage, videó/hang támogatás) a fejlettebb funkciók érdekében [12, 21, 22]. Ennek eredményeként a modern weboldalak képesek összetett logikát futtatni a böngészőben, offline gyorsítótárazást és push értesítéseket használni [23]. A „statikus weboldal” és a „webalkalmazás” közötti különbség elmosódott, mivel még a tartalomközpontú oldalak is átvették a kliensoldali renderelést [24, 17].

1.5. Mobil és web standardok (reszponzív dizájn)

Az okostelefonok korszaka újratervezésre kényszerítette a webalkalmazásokat [25]. Az olyan webes szabványok, mint a CSS3 media query-k és a flexibilis elrendezések, lehetővé tették a reszponzív dizájnt: olyan oldalak létrehozását, amelyek alkalmazkodnak a különböző képernyőméretekhez [26, 27]. A W3C HTML5 és CSS3 specifikációi támogatást nyújtottak mobilbarát funkciókhoz is (pl. érintőképernyő általi események) [12, 28].

Sok weboldal átvette a „mobile-first” tervezési szemléletet [29]. A körülbelül 2015-ben megjelent Progressive Web Appok (PWA-k) Service Workereket és manifest fájlokat használnak annak érdekében, hogy natív alkalmazásokhoz hasonló élményt nyújtsanak (offline mód, telepíthető ikonok) [30, 31]. Ezek a modern HTML5/JavaScript technológiákra és böngésző API-kra (kamera, helymeghatározás stb.) épülnek, hogy fejlett funkcionalitást biztosítsanak [32, 33].

1.6. Jelenlegi állapot és trendek

A mai webes technológiai stackek gyakran mikroszolgáltatásokat (API-kon keresztül kommunikáló, elkülönült szolgáltatásokat) használnak, amelyeket sokszor szerver nélküli környezetben (serverless) futtatnak, például az AWS Lambda szolgáltatásával, amelyet 2014-ben vezettek be [34].

A biztonság és az adatvédelem kiemelten fontossá vált: a HTTPS mára szinte mindenhol elterjedt [35]. A Let's Encrypt automatizálta a tanúsítványok kiadását, és a böngészők egyre inkább mindenhol titkosítást követelnek meg [36, 37]. A tartalomszolgáltató hálózatok (CDN-ek) és az edge computing javítják a teljesítményt [38].

A fejlesztési folyamatok iteratívvá váltak: az Agile/DevOps csapatok gyakori frissítéseket telepítenek, és automatizált tesztelési eszközöket valamint monitorozási megoldásokat használnak [39, 6]. A webes szabványok folyamatosan fejlődnek, miközben a WHATWG és a W3C a HTML, az Ecma International pedig a JavaScript alapját adó ECMAScript továbbfejlesztésén dolgozik [40, 41]. A böngészők új technológiákkal (például WebAssembly) bővítik a platformot [42].

A webes és a natív alkalmazások közötti határ egyre inkább elmosódik, ahogy a PWA-k egyre népszerűbbé válnak [31]. A hibrid keretrendszerek és platformok, mint a React Native vagy a Flutter web, szintén ezt a közeledést erősítik [43, 44].

1.7. A web fejlődésének rövid története - idővonal

1960-as évek

Az internet alapjai

Az ARPANET és a csomagkapcsolt adatátvitel megjelenése megalapozta a későbbi, egymással összekapcsolt számítógépes hálózatokat [45, 46].

1989–1991

A World Wide Web megszületése

Tim Berners-Lee a CERN-ben kidolgozta a World Wide Web koncepcióját, majd megjelentek a web alapvető elemei: a HTML, a HTTP, az URL, az első webszerver és az első böngésző [1, 8].

1990-es évek

A statikus web korszaka

A korai weboldalak főként HTML-alapú, információközlő dokumentumok voltak, amelyeket a webszerverek dinamikus tartalomgenerálás nélkül szolgáltattak ki [9]. A grafikus böngészők, különösen az NCSA Mosaic, jelentősen hozzájárultak a web elterjedéséhez [10].

2000-es évek

Dinamikus webalkalmazások

A szerveroldali programozás és az adatbázisok használata lehetővé tette a dinamikus tartalom előállítását [2]. Az AJAX elterjedése interaktívabb, részleges frissítésre képes webes felületeket eredményezett [3, 15].

2010-es évek

Modern kliensoldali web

Az egyoldalas alkalmazások és a frontend keretrendszerek a böngészőt egyre inkább alkalmazásfuttatási környezetté tették [18, 17]. Ezzel párhuzamosan a felhőalapú infrastruktúra és a DevOps szemlélet meghatározóvá vált a webalkalmazások fejlesztésében és üzemeltetésében [5, 6].

Webalkalmazások napjainkban

A PWA-k és a szerver nélküli architektúrák tovább közelítették egymáshoz a webes és natív alkalmazások felhasználói élményét [31, 34]. A mesterséges intelligencia és az automatizáció új fejlesztési és tartalom-előállítási irányokat nyitott a weben [47].

2. fejezet

Szakirodalom áttekintése

2.1. Cím 1

2.1.1. Alcím 1

Táblázat:

Kritériumok	Vrep	ARGoS	Gazebo
Ingyenes	Igen, van fizetős verzió is	Igen	Igen
Absztrakciós szint	Valósághű	Emelkedett absztrakciós szintet ajánl	Valósághű
Robotrajokra optimalizált	Nem optimalizált	Teljesen optimalizált	Képes, nagyobb erőforrás-igény, mint az ARGOS-nak
Nyílt forráskódú	Igen	Igen	Igen
Támogatott programozási nyelvek	C/C++, Python, Java, Lua, Matlab, Octave	C/C++ és Lua	C/C++
Valós robotok modelljei	Igen	Igen	Igen

2.1. ábra. V-REP, ARGoS, Gazebo összehasonlítása

Hivatkozás a táblázatra: 2.1.1

3. fejezet

Elméleti áttekintés

3.1. Elméleti áttekintés

Pszudokód:

Data: Tanulási tényező ($\alpha \in (0, 1]$), $\varepsilon > 0$

Véletlenszerű érték minden $Q_1(s, a)$ és $Q_2(s, a)$ -nek, kivéve $Q(\text{terminális}, \cdot) = 0$, $s \in S$, $a \in A$;

for minden *epizód* **do**

 S inicializálása;

repeat

$A \leftarrow$ cselekvés, S állapotban ε -greedy szerint $Q_1 + Q_2$;

A cselekedet végrehajtása, R és S' megfigyelése;

if 50% *eséllyel* **then**

$Q_1(S, A) \leftarrow Q_1(S, A) + \alpha[R + \gamma Q_2(S', \arg \max_a Q_1(S', a)) - Q_1(S, A)]$

else

$Q_2(S, A) \leftarrow Q_2(S, A) + \alpha[R + \gamma Q_1(S', \arg \max_a Q_2(S', a)) - Q_2(S, A)]$

end

$S \leftarrow S'$;

until S *terminális állapot*;

end

Algorithm 1: Dupla Q-tanulás [48].

Hivatkozás pszeudokódra: 1.

4. fejezet

Rendszer specifikációi

4.1. Cím 1

5. fejezet

Gyakorlati megvalósítás

5.1. Ágensek vezérlése

Hivatkozásra példa

Az ágensek vezérléséhez a potenciálmező navigációs módszer volt felhasználva. Ez egy bevált módszer a robotrajok vezérléséhez [49]. Az alapötlete, hogy az akadályok taszító erővel hatnak az ágensre és a cél vonzó erővel. Ennek a két erőnek az eredője határozza meg az irányt amerre érdemes haladni.

5.1.1. Potenciálmező navigáció

Egyenletekre példa

A potenciálmező navigációs módszernél az erők nagysága az (5.1) egyenlet szerint van kiszámolva.

$$\begin{cases} |\vec{f}_{push}| = ae^{-\frac{(x-b_{push})^2}{2c_{push}^2}} \\ |\vec{f}_{pull}| = ae^{-\frac{(x-b_{pull})^2}{2c_{pull}^2}} \end{cases} \quad (5.1)$$

- a: Gauss görbe magassága
- b: Gauss görbe középpontja

- c: Gauss görbe szélessége

$$\vec{f}_{robot} = \sum_i \vec{f}_{push_i} + \sum_i \vec{f}_{pull_i} \quad (5.2)$$

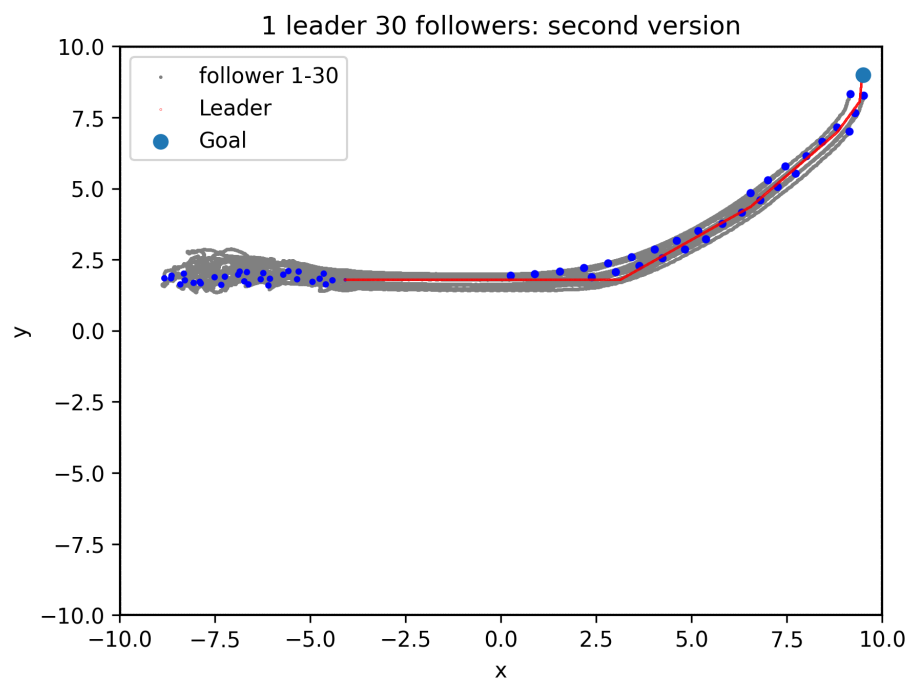
Az eredő vektor a (5.2) képlet szerint volt kiszámolva.

6. fejezet

Eredmények

6.1. Cím 1

Eredmények leírása



6.1. ábra. 30 követő ágens, egy vezér

7. fejezet

Összefoglalás

7.1. Összefoglalás

Irodalomjegyzék

- [1] CERN, „World Wide Web at 35.” <https://home.cern/world-wide-web-35/>, 2024. Accessed: 2026-05-28.
- [2] Mozilla Developer Network, „Introduction to the server side.” https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Server-side/First_steps/Introduction. Accessed: 2026-05-28.
- [3] J. J. Garrett, „Ajax: A New Approach to Web Applications.” https://www.oceanpark.com/webmuseum/2005/garrett_on_ajax.html, 2005. Accessed: 2026-05-28.
- [4] Mozilla Developer Network, „Responsive web design.” https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/CSS_layout/Responsive_Design. Accessed: 2026-05-28.
- [5] P. Mell and T. Grance, „The NIST Definition of Cloud Computing,” Tech. Rep. Special Publication 800-145, National Institute of Standards and Technology, 2011. <https://doi.org/10.6028/NIST.SP.800-145>. Accessed: 2026-05-28.
- [6] DevOps Research and Assessment, „Accelerate State of DevOps Report 2018.” <https://dora.dev/research/2018/dora-report/>, 2018. Accessed: 2026-05-28.
- [7] M. Fowler and J. Lewis, „Microservices.” <https://martinfowler.com/articles/microservices.html>, 2014. Accessed: 2026-05-28.
- [8] World Wide Web Consortium, „A Little History of the World Wide Web.” <https://www.w3.org/History.html>. Accessed: 2026-05-28.

- [9] Mozilla Developer Network, „What is a web server?.” https://developer.mozilla.org/en-US/docs/Learn_web_development/Howto/Web_mechanics/What_is_a_web_server. Accessed: 2026-05-28.
- [10] U.S. National Science Foundation, „Mosaic Launches an Internet Revolution.” <https://www.nsf.gov/news/mosaic-launches-internet-revolution>, 2004. Accessed: 2026-05-28.
- [11] World Wide Web Consortium, „The World Wide Web Consortium Issues HTML 4.0 as a W3C Recommendation.” <https://www.w3.org/press-releases/1997/html40-rec/>, 1997. Accessed: 2026-05-28.
- [12] World Wide Web Consortium, „HTML5.” <https://www.w3.org/TR/2014/REC-html5-20141028/>, 2014. Accessed: 2026-05-28.
- [13] D. Robinson and K. Coar, „The Common Gateway Interface (CGI) Version 1.1,” Tech. Rep. RFC 3875, RFC Editor, 2004. <https://doi.org/10.17487/RFC3875>. Accessed: 2026-05-28.
- [14] Mozilla Developer Network, „The web standards model.” https://developer.mozilla.org/en-US/docs/Learn_web_development/Getting_started/Web_standards/The_web_standards_model. Accessed: 2026-05-28.
- [15] Mozilla Developer Network, „Ajax.” <https://developer.mozilla.org/en-US/docs/Glossary/AJAX>. Accessed: 2026-05-28.
- [16] R. T. Fielding, M. Nottingham, and J. F. Reschke, „HTTP Semantics,” Tech. Rep. RFC 9110, RFC Editor, 2022. <https://doi.org/10.17487/RFC9110>. Accessed: 2026-05-28.
- [17] Mozilla Developer Network, „Introduction to client-side frameworks.” https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Frameworks_libraries/Introduction. Accessed: 2026-05-28.
- [18] Mozilla Developer Network, „SPA (Single-page application).” <https://developer.mozilla.org/en-US/docs/Glossary/SPA>. Accessed: 2026-05-28.

- [19] R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000. <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>. Accessed: 2026-05-28.
- [20] GraphQL Foundation, „GraphQL Specification, October 2021 Edition.” <https://spec.graphql.org/October2021/>, 2021. Accessed: 2026-05-28.
- [21] I. Fette and A. Melnikov, „The WebSocket Protocol,” Tech. Rep. RFC 6455, RFC Editor, 2011. <https://doi.org/10.17487/RFC6455>. Accessed: 2026-05-28.
- [22] World Wide Web Consortium, „Web Storage.” <https://www.w3.org/TR/2013/REC-webstorage-20130730/>, 2013. Accessed: 2026-05-28.
- [23] Mozilla Developer Network, „Offline and background operation.” https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps/Guides/Offline_and_background_operation. Accessed: 2026-05-28.
- [24] J. Wagner, „Client-side rendering of HTML and interactivity.” <https://web.dev/articles/client-side-rendering-of-html-and-interactivity>. Accessed: 2026-05-28.
- [25] World Wide Web Consortium, „Mobile Web Best Practices 1.0.” <https://www.w3.org/TR/2008/REC-mobile-bp-20080729/>, 2008. Accessed: 2026-05-28.
- [26] E. Marcotte, „Responsive Web Design.” <https://alistapart.com/article/responsive-web-design/>, 2010. Accessed: 2026-05-28.
- [27] World Wide Web Consortium, „Media Queries.” <https://www.w3.org/TR/2012/REC-css3-mediaqueries-20120619/>, 2012. Accessed: 2026-05-28.
- [28] World Wide Web Consortium, „Touch Events.” <https://www.w3.org/TR/2013/REC-touch-events-20131010/>, 2013. Accessed: 2026-05-28.
- [29] L. Wroblewski, *Mobile First*. A Book Apart, 2011. https://www.lukew.com/resources/mobile_first.asp. Accessed: 2026-05-28.

- [30] A. Russell, „Progressive Web Apps: Escaping Tabs Without Losing Our Soul.” <https://medium.com/@slightlylate/progressive-apps-escaping-tabs-without-losing-our-soul-3b93a8561955>, 2015. Accessed: 2026-05-28.
- [31] Mozilla Developer Network, „What is a progressive web app?.” https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps/Guides/What_is_a_progressive_web_app. Accessed: 2026-05-28.
- [32] Mozilla Developer Network, „MediaDevices: getUserMedia() method.” <https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getUserMedia>. Accessed: 2026-05-28.
- [33] World Wide Web Consortium, „Geolocation API.” <https://www.w3.org/TR/2024/REC-geolocation-20240613/>, 2024. Accessed: 2026-05-28.
- [34] Amazon Web Services, „Introducing AWS Lambda.” <https://aws.amazon.com/about-aws/whats-new/2014/11/13/introducing-aws-lambda/>, 2014. Accessed: 2026-05-28.
- [35] HTTP Archive, „Security: 2025 Web Almanac.” <https://almanac.httparchive.org/en/2025/security>, 2026. Accessed: 2026-05-28.
- [36] Let’s Encrypt, „Let’s Encrypt Launch Schedule.” <https://letsencrypt.org/2015/06/16/lets-encrypt-launch-schedule/>, 2015. Accessed: 2026-05-28.
- [37] E. Schechter, „A milestone for Chrome security: marking HTTP as "not secure".” <https://blog.google/products/chrome/milestone-chrome-security-marking-http-not-secure/>, 2018. Accessed: 2026-05-28.
- [38] Cloudflare, „What is a CDN edge server?.” <https://www.cloudflare.com/learning/cdn/glossary/edge-server/>. Accessed: 2026-05-28.
- [39] K. Beck, M. Beedle, A. van Bennekum, A. Cockburn, W. Cunningham, M. Fowler, J. Grenning, J. Highsmith, A. Hunt, R. Jeffries, J. Kern, B. Marick, R. C. Martin, S. Mellor, K. Schwaber, J. Sutherland, and D. Thomas, „Manifesto for Agile Software Development.” <https://agilemanifesto.org/>, 2001. Accessed: 2026-05-28.

- [40] World Wide Web Consortium and Web Hypertext Application Technology Working Group, „Memorandum of Understanding Between W3C and WHATWG.” <https://www.w3.org/2019/04/WHATWG-W3C-MOU.html>, 2019. Accessed: 2026-05-28.
- [41] Ecma International, „ECMAScript 2024 Language Specification.” <https://262.ecma-international.org/15.0/>, 2024. Accessed: 2026-05-28.
- [42] World Wide Web Consortium, „WebAssembly becomes a W3C Recommendation.” <https://www.w3.org/press-releases/2019/wasm/>, 2019. Accessed: 2026-05-28.
- [43] React Native Team, „Introduction.” <https://reactnative.dev/docs/getting-started>. Accessed: 2026-05-28.
- [44] Flutter Team, „Build apps for any screen.” <https://flutter.dev/>. Accessed: 2026-05-28.
- [45] Encyclopaedia Britannica, „ARPANET.” <https://www.britannica.com/topic/ARPANET>, 2026. Accessed: 2026-05-28.
- [46] Computer History Museum, „Internet History 1962 to 1992.” <https://www.computerhistory.org/internethistory/>. Accessed: 2026-05-28.
- [47] HTTP Archive, „Generative AI: 2025 Web Almanac.” <https://almanac.httparchive.org/en/2025/generative-ai>, 2026. Accessed: 2026-05-28.
- [48] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, 2018.
- [49] Z. Szántó, L. Márton, S. György, and T. I. Erdei, „Investigation of robotic swarms with partial team-goal knowledge,” in *2015 IEEE 19th International Conference on Intelligent Engineering Systems (INES)*, pp. 243–248, IEEE, 2015.

A. függelék

Függelék

A.1. Alfejezet

A.1.1. Cím

Alcím